Visitor Pattern for a File System

Olzhabekov Ali / SE-2515 / Astana IT University

01 / INTRODUCTION

One tree, several operations

This project implements Option A, File System Visitor, in Java 17. A file holds a name and a size; a directory holds a name and a list of children. The model is in memory. The application neither reads nor modifies operating-system files.

Size calculation and name search operate on the same tree. Their rules live in separate visitor classes, so adding an operation does not require adding another method to every element. A third visitor counts files and directories to demonstrate this extension.

```
project/
|-- README.txt        1200 bytes
|-- src/
|   |-- Main.java     2400 bytes
|   `-- notes.txt      600 bytes
`-- docs/
    `-- guide.txt     1800 bytes
```

Pattern role	Responsibility in this project
Element	FileSystemElement provides names, children and accept(visitor).
Concrete elements	FileNode and DirectoryNode hold data and dispatch to the matching visit method.
Visitor contract	FileSystemVisitor declares visitFile and visitDirectory.
Concrete visitors	SizeCalculatorVisitor sums bytes, SearchVisitor matches names, and ElementCountVisitor counts entries by kind.
Client and traversal	Main supplies one root to separate visitors. FileSystemWalker visits the root and its descendants in depth-first pre-order.

UML class diagram

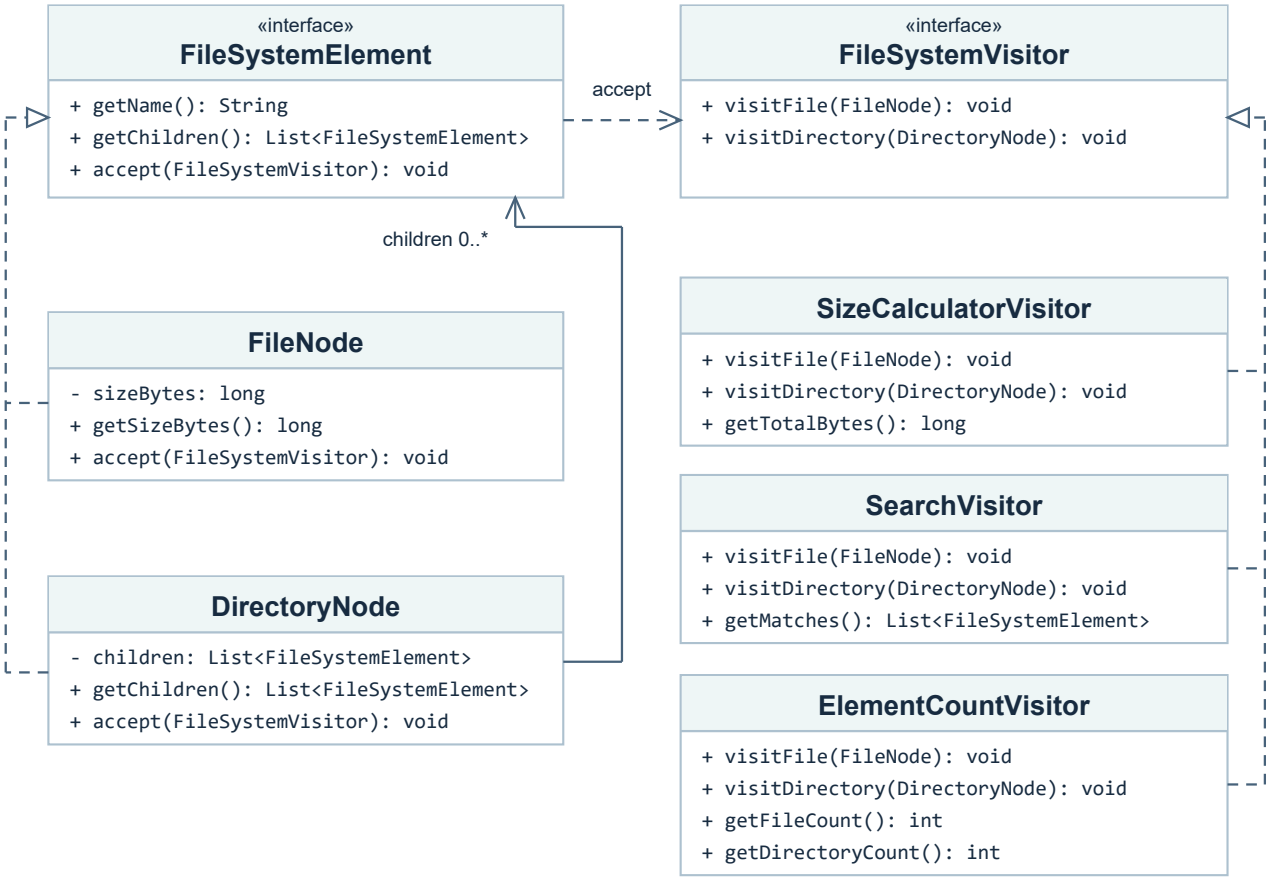


Figure 1. Dashed lines with hollow triangles show interface realization. The dashed accept arrow is a dependency; the solid children arrow is a navigable association. For readability, the diagram shows selected members. The editable PlantUML also includes the client classes and walker.

Double dispatch: from a file to its operation

```
// FileSystemWalker: the runtime element selects FileNode.accept.
element.accept(visitor);

// FileNode.accept: this is the current FileNode.
Objects.requireNonNull(visitor, "visitor").visitFile(this);

// SizeCalculatorVisitor.visitFile: the runtime visitor selects this body.
totalBytes = Math.addExact(totalBytes, file.getSizeBytes());
```

The first dynamic call selects the concrete element's accept method. Inside it, the element selects the correctly typed visit method and passes itself. The second dynamic call selects the implementation of that method on the concrete visitor. For a directory, the same traversal instead reaches visitDirectory. No type checks or casts are used to pick the operation.

Chapter 12: Emergence

The following excerpts are annotated to show how the four simple design rules apply to this implementation.

1. Keep behavior covered by tests

```
// VisitorTest: exercise both kinds of element through the same interface.
file.accept(visitor);
directory.accept(visitor);
equal(List.of("file:readme.txt", "directory:docs"), recorder.events);
```

The dispatch test checks observable callbacks through interface references. Other tests check the nested size total, search results, empty folders, overflow, defensive copies and the counting extension. These checks let the design change without losing behavior.

2. Put shared traversal in one place

```
// FileSystemWalker: every operation reuses this one traversal.
element.accept(visitor);
for (FileSystemElement child : element.getChildren()) {
    walk(child, visitor);
}
```

Size, search and counting do not each implement recursion. The walker owns visit order; a visitor owns its operation. Search likewise uses one private match method for both element kinds.

3. Make the operation obvious at the call site

```
// Main: the names identify the two operations and the common root.
SizeCalculatorVisitor size = new SizeCalculatorVisitor();
walker.walk(root, size);
// ...
SearchVisitor search = new SearchVisitor(".txt");
walker.walk(root, search);
```

A reader can see that the client calculates a size and searches by name. The names visitFile and visitDirectory also make the destination of each accept call explicit.

4. Keep the visitor contract small

```
// FileSystemVisitor: one required method per supported element kind.
void visitFile(FileNode file);
void visitDirectory(DirectoryNode directory);
```

The interface does not impose result types, reset methods or traversal hooks on every operation. Only concrete visitors expose the results they produce.

Chapter 13: Concurrency

5. Copy data at the boundary

```
// DirectoryNode constructor: retain a read-only snapshot of children.
this.children = List.copyOf(children);

// SearchVisitor.getMatches: callers cannot edit the result collector.
return List.copyOf(matches);
```

A caller can change its original input list without changing directory membership. Search returns a read-only snapshot, so later visits cannot modify a previously returned list. These copies are shallow: safe sharing here relies on the supplied FileNode and DirectoryNode implementations being immutable.

6. Confine mutable visitor state to one traversal

```
// VisitorTest: construct the mutable visitor inside a worker task.
var size = executor.submit(() -> {
    SizeCalculatorVisitor visitor = new SizeCalculatorVisitor();
    walker.walk(root, visitor);
    return visitor.getTotalBytes();
});
```

The test runs size and search in separate tasks using one immutable root and a stateless walker. Each task creates its own visitor. The byte total and the list of matches therefore belong to different tasks. A single mutable visitor must not be shared across threads; create a new instance for a new result.

Extension: one new operation

```
// ExtensionDemo: the same walker and element API support a new visitor.
FileSystemElement root = SampleFileSystem.create();
ElementCountVisitor counts = new ElementCountVisitor();
new FileSystemWalker().walk(root, counts);
```

ElementCountVisitor increments one counter in visitFile and the other in visitDirectory. The extension commit adds only the visitor and a new demo in production code; it also adds tests. FileNode, DirectoryNode, both interfaces and the walker are unchanged.

Run results and conclusion

Main demo: size and search on the same tree

```
Same file system: project (4 files, 3 directories)
Total file-content size: 6000 bytes
Names containing .txt:
- README.txt
- notes.txt
- guide.txt
Names containing docs:
- docs
All visitors used the same root; no element was changed.
```

Extension demo: a new visitor

```
New operation on the same sample tree: project
File count: 4
Directory count (including root): 3
```

Verification. All 20 behavioral tests passed on the Java 17 runtime. They cover both dispatch methods, traversal order, name and size results, immutable collections, isolated parallel traversals, and the new visitor.

Encapsulation and trade-off

Elements keep their state private and expose only read access needed by the operations. The walker owns traversal; each visitor owns its mutable result. This separates the reasons for changing the model and the operations. A new operation can be added as another visitor, while a new element type generally requires a new method in FileSystemVisitor and updates to all existing visitors. An operation that needs previously hidden data may also require an intentional change to the element API.

Repository and references

GitHub: https://github.com/maxiszero/assignment-6-visitor-pattern

Course materials: Assignment 6 instructions, Option A, and the LMS note for Clean Code Chapters 12 and 13.

Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*, Pearson, 2008, Chapters 12 and 13. Publisher and table of contents.